Mathematical Modeling Lecture Notes
Chapter 4 – Experimental Modeling

I. Background
   a. We've spent some time discussing how to fit the parameters of a model, assuming we have some argument as to why the model is appropriate.
   b. Such models can be used to make predictions, both inside (interpolation) and outside (extrapolation) the range of input data values.
   c. In the absence of some argument for why a particular model is appropriate, we can still do interpolation, but any kind of extrapolation would be very suspect.

II. Polynomial interpolation
   a. Background
      i. We've already looked at power models, like $y = ax^p$, where p is any real number.
      ii. We'll now look at more complicated polynomial models in which $y = a_0 + a_1x + \cdots + a_px^p$.
   b. Theorem: If $x_0, x_1, \ldots, x_n$ are $n+1$ distinct points, and $y_0, y_1, \ldots, y_n$ are the corresponding observations at these points, then there exists a unique polynomial $y = P(x)$ of degree n or less with the property that $P(x_i) = y_i$ for each i.
      i. Proof of uniqueness
         1. Suppose there are two such polynomials P and Q.
         2. Let $F(x) = P(x) - Q(x)$.
         3. Then $F(x_i) = 0$ for all i, $0 \le i \le n$.
         4. Since the degree of F can be at most n it is either the zero polynomial or has at most n zeros.
         5. However, since the x_i are all distinct F has at least the $n+1$ zeros $x_0, x_1, \ldots, x_n$.
         6. Thus F must be the zero polynomial, and $P(x) = Q(x)$.
      ii. Proof of existence
         1. If $n = 0$ then the constant polynomial $P(x) = y_0$ has $P(x_0) = y_0$.
         2. Now suppose we have a polynomial $P_{k-1}(x)$ of degree no larger than $k-1$ such that $P_{k-1}(x_i) = y_i$ for all i, $0 \le i \le k-1$.
         3. Let

$$P_k(x) = P_{k-1}(x) + \frac{(y_k - P_{k-1}(x_k))(x - x_0)(x - x_1)\cdots(x - x_{k-1})}{(x_k - x_0)(x_k - x_1)\cdots(x_k - x_{k-1})}.$$

         4. Then $P_k(x_i) = y_i$ for all i, $0 \le i \le k$.
   c. Lagrangian form
      i. There are different forms for expressing the interpolating polynomial.
      ii. The *Lagrangian form* is given by $P(x) = y_0L_0(x) + \cdots + y_nL_n(x)$, where

$$L_k(x) = \frac{(x - x_0)(x - x_1)\cdots(x - x_{k-1})(x - x_{k+1})\cdots(x - x_n)}{(x_k - x_0)(x_k - x_1)\cdots(x_k - x_{k-1})(x_k - x_{k+1})\cdots(x_k - x_n)}.$$

      iii. The polynomial can be (relatively) easily calculated with this form.
   d. How to determine the interpolating polynomial for the stopping distance data
      i. We should have a degree 16 polynomial for the stopping distance data.
      ii. This polynomial can be defined as $P(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{16}x^{16}$.

iii. In order to determine the polynomial, then, we need to determine the seventeen coefficients involved.

iv. We have seventeen data points, which can be used to give us seventeen linear equations with sixteen unknowns.

$$\begin{aligned} 0 &= a_0 + a_1(0) + a(2)(0)^2 + \cdots + a_{16}(0)^{16} \\ 3 &= a_0 + a_1(1) + a_2(1)^2 + \cdots + a_{16}(1)^{16} \\ 6 &= a_0 + a_1(2) + a_2(2)^2 + \cdots + a_{16}(2)^{16} \\ &\vdots \\ 376 &= a_0 + a_1(16) + a_2(16)^2 + \cdots + a_{16}(16)^{16} \end{aligned}$$

v. This would be a pain to solve, but it can be done with some matrix inversion.
   1. How do we know the underlying matrix is invertible?
   2. Basically, we're dealing with consecutive powers of consecutive integers, which cannot be expressed as a linear combination of each other.

e. Use Python to calculate the interpolating polynomial of degree 16 for the stopping distance data.

f. Advantages and disadvantages

i. Advantages
   1. The polynomial fits the data values exactly.
   2. Can be good for making predictions within the range of observed x values.
   3. Polynomials are nice theoretically (e.g., can be easily differentiated and integrated).

ii. Disadvantages
   1. They are often very bad for making predictions outside the range of observed x values.
   2. In particular, they tend to oscillate badly near the ends of the range of observed data.
   3. They can be very sensitive to small changes in data values. (See example in text.)

III. Smoothing

a. Be careful with overfitting a model.

i. You can use a high-order polynomial to fit a dataset so that all the deviations are zero.

ii. However, it might be better, because of some of the disadvantages of polynomial interpolation, to use a lower-order polynomial with a higher MAD or sum of squares.

iii. How do you choose an appropriate lower-order polynomial, though?

b. Training/validation

i. One way to handle this, if you have enough data, is to split it into a training set and a validation set.
   1. You train several models on the training set.
   2. Then you validate them all on the validation set.
   3. Then you take which one is the best.

ii. Consider the ponderosa pine data, where x represents the diameter of a tree measured at about 5 feet and y represents the total number of board feet (divided by 10) of lumber obtained from it. (Plot training data first.)

1. Training

x	17	19	20	22	23	25	28	31	32	33	36	37	39	42
y	19	25	32	51	57	71	113	140	153	187	192	205	250	260

2. Validation

x	17	17	19	19	25	25	28	31	32	38	39	39	41
y	16	22	28	21	56	86	88	141	123	252	231	265	294

3. Show Python code that trains and then validates a variety of polynomials on the data.
4. Best models
   - Linear: $y = -17.7 + 10.4x$
   - Quadratic: $y = -111.0 + 5.6x + 0.08x^2$
   - 8th order: $y = -328880 + 102472x - 13775x^2 + 1043x^3 - 49x^4 + 1.4x^5 - 0.026x^6 + 0.00027x^7 - 0.0000012x^8$.
5. Decision
   - While the 8th order model does best on the validation set, it's really unintuitive.
   - Quadratic model doesn't seem to be much better than the linear one.
   - It seems reasonable to go with the linear model.

IV. Splines

a. A *spline function* consists of piecewise polynomials and is also used for interpolation.

b. Linear

i. Linear splines consist of piecewise linear polynomials.

ii. For example, let's fit a linear spline function to the stopping distance data.

1. Python's `plot` function automatically uses a linear spline. (Show this.)
2. If you want the value of the spline at a specific point, you can use the `interp1d` function in the `scipy.interpolate` library.
3. Show Python code.
4. Mathematically, the equation for the piece of the spline between the points (4,21) and (5,32) can be found simply by finding the line between those two points.
   - The slope is (32-21)/(5-4) = 11.
   - Thus this piece of the spline is $\hat{y} = 11(x - 4) + 21$, or $\hat{y} = 11x - 23$, $4 \leq x \leq 5$.
   - And so the prediction is $\hat{y}(4.5) = 11(4.5 - 4) + 21 = 26.5$.
5. The other pieces of the spline function can be found in a similar fashion.
6. As with the other methods in Chapter 4, the spline function can be used to predict stopping distances for speeds within the observed range of data.
   - It is used for interpolation, not extrapolation.

iii. While continuous, linear splines are not continuously differentiable.

c. Cubic

i. Cubic splines, which fit cubic functions between data points, are a little trickier.

ii. They yield functions that have continuous first and second derivatives (and thus have continuous curvature and so appear smooth).

iii. Conditions

1. A generic cubic function is of the form $y = A + Bx + Cx^2 + Dx^3$.
2. If there are n data points then there are $n - 1$ piecewise cubic functions we have to find.
3. Thus there are a total of $4(n - 1) = 4n - 4$ parameters to be obtained.

4. By requiring each piece to go through its two endpoints we have $2(n-1) = 2n-2$ conditions.
   - Mathematically, this is $P_i(x_i) = y_i$ and $P_i(x_{i+1}) = y_{i+1}$ for $1 \le i \le n-1$.
5. By requiring the derivatives to match up at each interior endpoint of the interval we have $n-2$ additional conditions, for a total of $3n-4$ conditions.
   - $P'_{i-1}(x_i) = P'_i(x_i)$ for $2 \le i \le n-1$.
6. By requiring the second derivatives to match up at each interior endpoint we have $n-2$ more conditions, for a total of $4n-6$ conditions.
   - $P''_{i-1}(x_i) = P''_i(x_i)$ for $2 \le i \le n-1$.
7. From where do we get the remaining two conditions?
   - There are generally three options.
   - *Natural splines* require the first derivatives to be constant at the exterior endpoints.
     - This corresponds to the curve continuing in the direction that it had been going.
     - We implement this by requiring the second derivatives to be zero at the exterior endpoints.
   - *Clamped splines* require the first derivatives to fit some known values at the exterior endpoints.
     - This option should only be used when you have known values for the first derivatives at the exterior endpoints.
   - *Not-a-knot splines* require the third derivatives at points x_2 and x_{n-1} to match up.
     - By this we mean $P'''_1(x_2) = P'''_2(x_2)$ and $P'''_{n-2}(x_{n-1}) = P'''_{n-1}(x_{n-1})$.
     - This has the effect of forcing $P_1 = P_2$ and $P_{n-2} = P_{n-1}$, as specifying f, f', f'', f''' completely specifies a cubic function.
8. With $4n-4$ conditions and $4n-4$ unknowns we can set up a system of linear equations to solve for the coefficients we need.
9. What does Python's `interp1d` function do?
   - Python's `interp1d` default is to use not-a-knot splines.
   - However, there are some other Python spline functions you can use to force natural or clamped splines if you really want them.
   - For our purposes, I'd just use the default not-a-knot spline.

iv. Show stopping distance cubic spline to students in Python..

v. Show ponderosa pine data with the cubic vs. linear splines.

V. Show modeling flowchart on p. 179. (Show scanned .pdf.)

Homework:

- 4.1 #1-4 (give to same person)
- 4.2 #4
- 4.3 #5, 8. For #5 and #8, follow the instructions except that instead of using divided differences, use a random number generator to split the data into a training set and a validation set. Train several low-order polynomials on the training set data. Then use the validation set to select a low-order polynomial that makes sense as a model.
- 4.4 #1a (make sure you solve the system of equations), Projects 6 and 7 (use Python to find the splines. Print out the plots, together with the original data. You don't have to write down the actual spline equations).